

● ARCHITECTURE & IMPLEMENTATION GUIDE

# Tokenboard / Copilot Edition

*Recreate the usage-analytics dashboard for Windows + GitHub Copilot.*

A complete blueprint to rebuild the app from scratch — data sources, backend, frontend, AI insights, packaging, and CI — targeting Windows only and GitHub Copilot only.

## PLATFORM

### Windows 10/11 only

Single MSI / EXE installer · WebView2 · no macOS, no Linux

## AI SOURCE

### GitHub Copilot only

VS Code chat sessions + Copilot CLI · no Claude, no API keys

## STACK

### Tauri 2 · Rust · HTML

~10 MB native app · Chart.js dashboard · 5 tabs

Prepared for the Tokenboard rebuild · June 2026

Audience: a developer recreating the app with no access to the original macOS/Claude codebase

Self-contained: every data contract, module, and command needed to ship v1 is in this document

## Contents

---

|    |                                                           |
|----|-----------------------------------------------------------|
| 1  | Purpose, scope & what changes from the original           |
| 2  | The proven architecture we are porting                    |
| 3  | Target architecture (Windows + Copilot)                   |
| 4  | Data sources on Windows — where Copilot keeps its history |
| 5  | The data contract — backend → frontend JSON               |
| 6  | Backend: the Rust analyzer                                |
| 7  | AI insights via the Copilot CLI                           |
| 8  | Frontend: the five-tab dashboard                          |
| 9  | Tauri shell & Windows packaging                           |
| 10 | CI/CD — GitHub Actions, Windows runner                    |
| 11 | Build & run — exact commands                              |
| 12 | Risks, unknowns & a validation checklist                  |
| A  | Appendix — file tree & dependency manifest                |

### How to read this document

Sections **5–7** are the load-bearing engineering content: the JSON contract you can reuse almost verbatim, the analyzer pipeline, and the Copilot-CLI integration. Sections **4** and **12** are the honest part — the Copilot on-disk format is *not* officially documented and shifts between VS Code versions, so the design is built around a **discovery-first** approach rather than a frozen schema. Treat field names in Copilot data as things to confirm on the target machine, not as guarantees.

# 1 · Purpose, scope & what changes

Tokenboard reads the local history an AI coding assistant leaves on disk and turns it into a private dashboard — where time and requests go, which projects cost the most, what habits to fix. The original parses Claude Code transcripts on macOS. This edition targets **Windows only** and **GitHub Copilot only**.

## 1.1 What stays identical

- **The shape of the product.** A Tauri desktop app: a Rust backend parses local files into one JSON blob; a static HTML/Chart.js frontend renders five tabs (Overview, Tokens/Usage, Repos & Sessions, Tools & Models, Rhythm).
- **The data contract.** The backend → frontend JSON (Section 5) is kept stable so the frontend ports with near-zero change.
- **The analytics.** Per-repo and per-session rollups, daily series, hour/weekday histograms, efficiency ratios, and the rule-based "insight cards."
- **The AI-lessons feature.** One click mines real failures from history and asks the assistant's own CLI to write project-specific conventions — riding the user's existing subscription, no API key.
- **Privacy posture.** Everything is read locally and stays local. No telemetry, no network calls except the user-initiated Copilot CLI call.

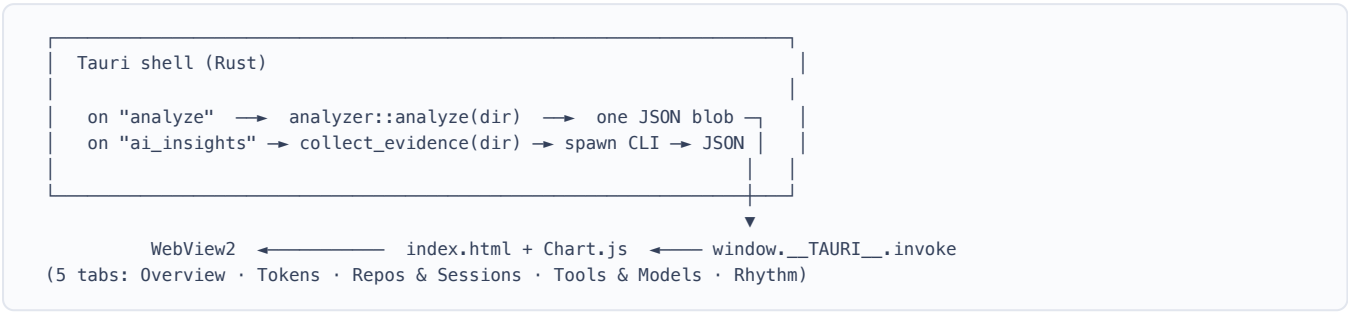
## 1.2 What changes — and why

| CONCERN        | ORIGINAL (CLAUDE CODE / MACOS)                                   | THIS EDITION (COPILOT / WINDOWS)                                                                                                |
|----------------|------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Data location  | <code>~/.claude/projects/**/*.jsonl</code>                       | <code>%APPDATA%\Code\User\workspaceStorage\<br/>&lt;hash&gt;\chatSessions\*.json</code> + Copilot CLI session store (Section 4) |
| Record format  | One JSON object per line (JSONL), stable documented-ish fields   | Per-session JSON files; <b>schema undocumented &amp; version-dependent</b> — must be probed                                     |
| Cost model     | Per-token pricing (input/output/cache) per model                 | Copilot meters <b>"premium requests"</b> with per-model multipliers, not tokens. Token counts may be absent. (Section 6.4)      |
| "Tool calls"   | Explicit <code>tool_use</code> / <code>tool_result</code> blocks | Agent-mode tool invocations in Copilot CLI/VS Code logs — present but named differently                                         |
| AI-lessons CLI | <code>claude -p --output-format json</code>                      | <code>copilot -p "..." -s</code> (programmatic mode) (Section 7)                                                                |
| Process spawn  | <code>/bin/zsh -lc</code>                                        | <code>cmd /C / powershell</code> — resolve <code>copilot.cmd</code> on PATH (Section 7.3)                                       |
| Packaging      | DMG + ad-hoc codesign; also MSI/deb/rpm                          | MSI (WiX) + NSIS <code>.exe</code> only; WebView2 runtime; unsigned → SmartScreen note                                          |
| CI matrix      | 4-way (2× macOS, Windows, Linux)                                 | Single <code>windows-latest</code> job                                                                                          |

**Design principle.** Because the Copilot on-disk format is unofficial, the backend is split into a *stable core* (rollups, insights, output JSON — ported verbatim) and a thin, replaceable **ingest adapter** that maps Copilot's fields onto the core's internal model. When VS Code changes its format, only the adapter changes.

## 2 · The proven architecture we are porting

The original is ~1,000 lines across three files. Understanding its data flow is the fastest route to rebuilding it.



### 2.1 The three files

| FILE                      | ROLE                                                                                                                                                                                        | LINES |
|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| src-tauri/src/analyzer.rs | Walks the data dir, folds every record into per-session / per-repo / per-model / daily / histogram accumulators, computes efficiency ratios and rule-based insights, emits one JSON object. | ~800  |
| src-tauri/src/main.rs     | Two Tauri commands ( analyze , ai_insights ) + two CLI debug modes ( --dump , --ai ). Spawns the assistant CLI for lesson mining.                                                           | ~130  |
| src/index.html            | Self-contained dashboard: pulls the JSON via invoke( 'analyze' ) , renders KPI cards, Chart.js charts, sortable tables, the hour/weekday heat strip, and the insight cards. Five tabs.      | ~400  |

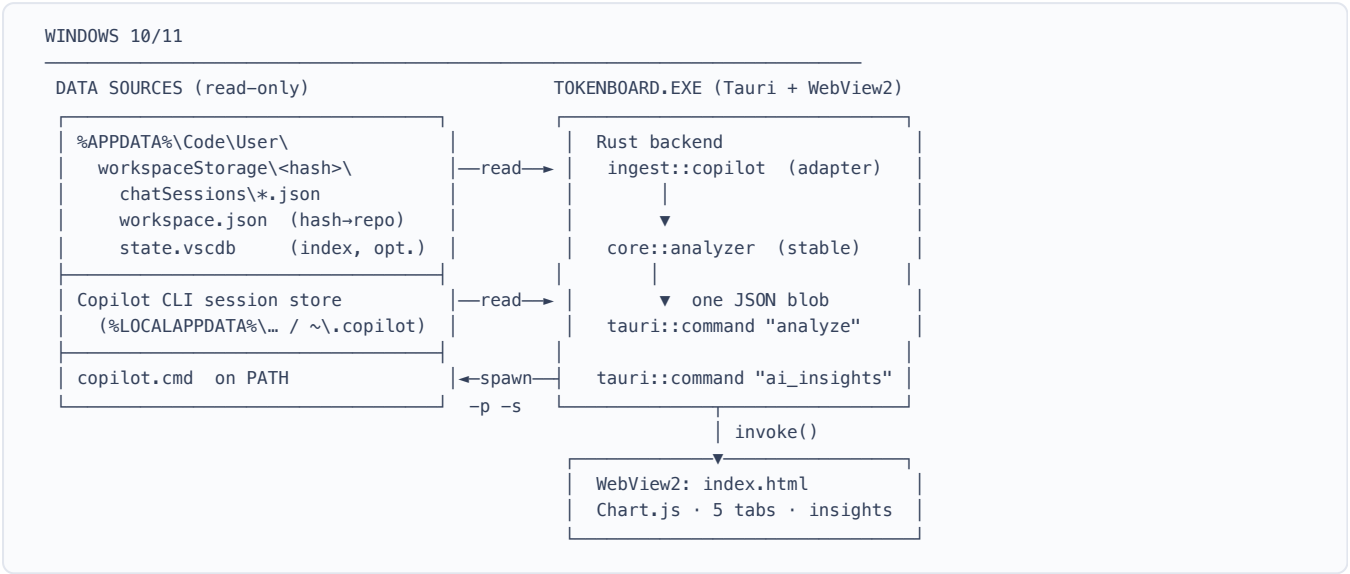
### 2.2 Two-pass parse (worth preserving)

The original parses **main sessions** first, then a second pass folds **subagent side-transcripts** into their parent session. The Copilot edition keeps the two-pass *structure* because Copilot likewise has a primary chat plus agent/tool sub-activity; pass 2 simply reads a different sub-source (or is a no-op in v1 — Section 6.6).

### 2.3 The "active vs idle" heuristic

Time is derived purely from message timestamps within a session: consecutive gaps ≤ 5 min count as **active**; gaps of 5–30 min count as **idle** (stepped-away); gaps > 30 min are ignored (session boundary). This is data-source-agnostic and ports unchanged — *provided* each Copilot message carries a timestamp (verify in Section 4).

### 3 · Target architecture (Windows + Copilot)



#### 3.1 Module layout (Rust)

| MODULE                         | STABILITY   | RESPONSIBILITY                                                                                                                                                                    |
|--------------------------------|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>core/model.rs</code>     | Stable      | <code>Stats</code> , <code>Session</code> , <code>ModelAgg</code> , <code>DailyAgg</code> , accumulators — ported verbatim from the original.                                     |
| <code>core/analyzer.rs</code>  | Stable      | Rollups, efficiency ratios, <code>build_insights()</code> , output-JSON shaping — ported verbatim.                                                                                |
| <code>ingest/copilot.rs</code> | REPLACEABLE | Discovers Copilot data dirs, reads session files, maps Copilot fields → <code>core events (UserTurn, AssistantTurn, ToolCall, ToolError)</code> . The only Copilot-specific code. |
| <code>ingest/probe.rs</code>   | Tool        | <code>--probe</code> mode: dumps the distinct JSON keys/shapes found in the user's session files so the adapter can be calibrated on the real machine.                            |
| <code>main.rs</code>           | Thin        | Tauri commands + CLI modes; Windows path resolution; Copilot CLI spawn.                                                                                                           |

**The single most important decision:** keep `core` byte-for-byte faithful to the original and isolate *all* Copilot knowledge in `ingest/copilot.rs`. That makes the frontend, the analytics, and the insight rules a known-good port, and concentrates the genuine unknown (Copilot's format) in one ~150-line file you can iterate on with `--probe`.

## 4 · Data sources on Windows

GitHub Copilot leaves two distinct local histories. Tokenboard reads both, normalizes them, and de-duplicates by session id.

### 4.1 Source A — VS Code Copilot Chat sessions

| PATH                                                                | CONTENTS                                                                                                                                                                                |
|---------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| %APPDATA%\Code\User\workspaceStorage\<br><hash>\chatSessions\*.json | One file per chat session. Each holds the full conversation (user requests, assistant responses, and — in agent mode — tool invocations). Reported to be self-contained JSON/JSONL.     |
| %APPDATA%\Code\User\workspaceStorage\<br><hash>\workspace.json      | Maps the opaque <hash> to the original workspace folder URI — this is how you recover the <b>repo/project name</b> for a session (the analog of the original's <code>cwd</code> field). |
| %APPDATA%\Code\User\workspaceStorage\<br><hash>\state.vscdb         | SQLite index (optional). Useful as a fallback to enumerate sessions; not required if you glob the JSON files.                                                                           |

VS Code Insiders uses `Code - Insiders` in place of `Code`. Also probe `%APPDATA%\Code\User\globalStorage\github.copilot-chat\` for newer builds that moved transcripts there. Enumerate, don't hardcode the single path.

### 4.2 Source B — GitHub Copilot CLI sessions

The standalone Copilot CLI (GA since 2025) keeps its own agent-session history. Its location is not formally documented; probe, in order:

|                                                          |                                                                       |
|----------------------------------------------------------|-----------------------------------------------------------------------|
| %LOCALAPPDATA%\github-copilot\<br>%USERPROFILE%\copilot\ | %APPDATA%\GitHub Copilot CLI\<br>%USERPROFILE%\config\github-copilot\ |
|----------------------------------------------------------|-----------------------------------------------------------------------|

The CLI can also **export** a session to Markdown with `--share '<path>'` — a stable, documented escape hatch if the raw store proves hard to parse. v1 may rely on Source A alone and treat Source B as additive.

### 4.3 The field-mapping table — what the core needs vs. where to find it

The core analyzer consumes a small set of **normalized events**. The adapter's whole job is to populate them from whatever Copilot actually writes. Confirm each right-hand cell with `--probe` on a real machine before trusting it.

| CORE NEEDS        | ORIGINAL CLAUDE FIELD                     | LIKELY COPILOT LOCATION (VERIFY)                                                                                           |
|-------------------|-------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| session id        | file stem                                 | chatSession file stem / <code>sessionId</code> / <code>requesterUsername +id</code>                                        |
| repo / project    | <code>cwd</code>                          | <code>workspace.json</code> → folder URI → leaf folder name                                                                |
| message timestamp | timestamp (RFC3339)                       | per-message <code>timestamp</code> / <code>createdAt</code> (epoch ms?) — <b>critical for time + rhythm</b>                |
| user turn         | <code>type:"user"</code> , string content | request object: <code>message.text</code> / <code>request.message</code>                                                   |
| assistant turn    | <code>type:"assistant"</code>             | response object: <code>response[]</code> / <code>result</code>                                                             |
| model             | <code>message.model</code>                | <code>modelId</code> / <code>request.modelId</code> (e.g. <code>gpt-4o</code> , <code>claude-sonnet-4</code> )             |
| token usage       | <code>usage.*_tokens</code>               | <b>often absent</b> — see 6.4; fall back to request counts                                                                 |
| tool call         | <code>tool_use</code> block               | agent steps: <code>toolInvocations[]</code> / <code>toolCallRounds[]</code> / <code>kind:"toolInvocationSerialized"</code> |
| tool error        | <code>tool_result.is_error</code>         | tool result with <code>isError</code> / non-zero exit / error payload                                                      |
| title             | <code>type:"ai-title"</code>              | <code>customTitle</code> / first request text (truncated)                                                                  |

**Reality check.** The chatSessions JSON schema is internal to the VS Code Copilot Chat extension and has changed repeatedly (community tools that read it break across releases). Do not ship a hardcoded path of field accesses. Ship the `--probe` mode first, run it on the target user's machine, and write the adapter against what you actually see. Section 12 has the validation checklist.

## 5 · The data contract (backend → frontend)

This is the single JSON object returned by the `analyze` command and consumed by the dashboard. **Keep these keys identical to the original** so the frontend ports unchanged. Reproduce it verbatim; only the *meaning* of cost/token fields shifts (Section 6.4).

```
{
  "generatedAt": "2026-06-20T15:52:24",
  "dataDir": "C:\\Users\\me\\AppData\\Roaming\\Code\\User\\workspaceStorage",
  "subagentFiles": 0,

  "totals": {
    // one global rollup → KPI cards
    "sessions":33, "repos":12, "userTurns":0, "assistantTurns":0,
    "toolCalls":0, "toolErrors":0,
    "inputTokens":0, "outputTokens":0,
    "cacheCreateTokens":0, "cacheReadTokens":0,
    "freshTokens":0, "totalTokens":0, "cost":0.0,
    "activeSeconds":0, "idleSeconds":0,
    "firstTs":null, "lastTs":null
  },

  "repos": [ { "name", "sessions", "userTurns", "assistantTurns",
    "toolCalls", "toolErrors", "inputTokens", "outputTokens",
    "cacheCreateTokens", "cacheReadTokens", "totalTokens",
    "cost", "activeSeconds", "idleSeconds",
    "branches":[], "firstTs", "lastTs" }, ... ], // sorted desc by totalTokens

  "models": [ { "model", "messages", "inputTokens", "outputTokens",
    "cacheCreateTokens", "cacheReadTokens", "cost" }, ... ], // desc by cost

  "tools": [ { "name", "count", "errors" }, ... ], // desc by count

  "daily": [ { "date":"2026-06-11", "totalTokens", "outputTokens",
    "userTurns", "cost", "activeSeconds" }, ... ], // chronological

  "sessions":[ { "id", "repo", "title", "firstTs", "lastTs",
    "userTurns", "toolCalls", "toolErrors", "totalTokens",
    "outputTokens", "cost", "activeSeconds", "idleSeconds" }, ... ],

  "efficiency": {
    "cacheHitRate", "outInRatio", "avgTokensPerTurn",
    "toolErrorRate", "costPerTurn", "cacheReadCostShare"
  },

  "hourHist": [ /* 24 ints, Mon-first local hour-of-day */ ],
  "dowHist": [ /* 7 ints, Mon..Sun */ ],

  "insights":[ { "severity":"warn|good|info", "title",
    "finding", "recommendation" }, ... ]
}
```

**Token fields under Copilot.** If Copilot logs expose no token counts, keep the keys (so the frontend and contract are unchanged) and populate them with `0` or with request-derived estimates. The Tokens tab then leads with *request volume* and *premium-request cost* rather than raw tokens. The frontend already tolerates zeros gracefully. See 6.4.

## 6 · Backend: the Rust analyzer

### 6.1 Internal model (port verbatim from the original)

The accumulators are data-source-agnostic. Reuse them exactly:

```
struct Stats { sessions, user_turns, assistant_turns, tool_calls, tool_errors,
               input_tokens, output_tokens, cache_create_tokens, cache_read_tokens,
               cost: f64, active_seconds: f64, idle_seconds: f64,
               first_ts, last_ts: Option<DateTime<Utc>>, branches: HashSet<String> }

struct Session { stats: Stats, title, cwd, repo,
                 tool_ids: HashMap<String,String>, // id → tool name
                 msgs: Vec<DateTime<Utc>>,         // for active/idle
                 tool_seq: Vec<String>,             // ordered tool ids (retry-loop rule)
                 err_ids: HashSet<String>,          // errored tool ids
                 turn_tokens: Vec<u64> }            // per-assistant-turn billed (bloat rule)

struct ModelAgg { messages, input_tokens, output_tokens,
                  cache_create_tokens, cache_read_tokens, cost }
struct DailyAgg { total_tokens, output_tokens, user_turns, cost, active_seconds }
```

### 6.2 The normalized-event seam

Instead of the original's `consume(json)` that knows Claude's field names, define a tiny event enum the adapter emits and the core folds. This is the entire contract between replaceable and stable code:

```
pub enum Event {
    UserTurn { ts: Option<DateTime<Utc>>, branch: Option<String> },
    Assistant { ts: Option<DateTime<Utc>>, model: String,
               usage: Usage, // zeros if unknown
               tools: Vec<(String /*id*/, String /*name*/)> },
    ToolError { id: String },
    Title { text: String },
}

pub struct Usage { input: u64, output: u64, cache_create: u64, cache_read: u64 }

// core: identical folding logic to the original consume(), minus JSON knowledge
fn fold(ev: Event, ss: &mut Session, a: &mut Analyzer, count_user: bool) { ... }
```

### 6.3 The adapter ( `ingest/copilot.rs` ) — the only new code

```
pub fn collect_sessions(roots: &[PathBuf]) -> Vec<(SessionId, Repo, Vec<Event>)> {
    for ws in enumerate_workspace_storage() { // %APPDATA%\Code\User\workspaceStorage\*
        let repo = repo_from_workspace_json(&ws); // hash → folder URI → leaf name
        for f in glob(ws.join("chatSessions\\*.json")) {
            let doc = read_json(f);
            // map Copilot fields → Event stream. Field paths come from --probe.
            for req in doc["requests"].as_array() {
                emit(UserTurn { ts: ts_of(req), branch: git_branch(req) });
                emit(Assistant { ts: ts_of(req), model: model_of(req),
                                usage: usage_of(req)/*maybe zero*/,
                                tools: tools_of(req) });
                for t in failed_tools(req) { emit(ToolError { id: t }); }
            }
        }
    }
}
```

Reuse the original's `repo_from_cwd` normalization (strip `file:///`, replace `\`, take the leaf) and its worktree-collapsing logic. The hour/weekday histograms, active/idle windows, and daily series all derive from `ts` alone — so they light up the moment timestamps map correctly.

### 6.4 Cost model: premium requests, not tokens **IMPORTANT**



GitHub Copilot does not bill per token; it meters **premium requests**, where each interaction with a premium model consumes `1 × multiplier` requests from a monthly allowance. Adapt the cost column accordingly:

- Replace `price_for(model) → [4 per-token prices]` with `multiplier_for(model) → f64`.
- **Notional "cost"** becomes **premium-request consumption** =  $\Sigma$  (assistant turns × model multiplier). Keep the JSON key `cost` so the frontend is unchanged; relabel it in the UI as "Premium requests."
- Populate the multiplier table from the *current* GitHub billing docs at build time — values change. Ship it as a small embedded JSON the adapter loads, not hardcoded constants.

| MODEL CLASS (EXAMPLE — CONFIRM AGAINST CURRENT GITHUB DOCS) | PREMIUM-REQUEST MULTIPLIER |
|-------------------------------------------------------------|----------------------------|
| Base model (included, e.g. GPT-4.1-class)                   | 0× (unmetered)             |
| Standard premium (e.g. Sonnet-class)                        | 1×                         |
| Higher-tier reasoning / Opus-class                          | ≥ 1× (often much higher)   |

**Do not hardcode the multipliers in this document into shipping code.** They are illustrative. Read them from GitHub's published billing page at build time and embed the resulting table. If token counts *are* present in the logs, keep the token KPIs too — they're strictly additive.

6.5 Insight rules — port all nine, retune two

The rule engine ( `build_insights` ) is pure functions over the accumulators and ports directly. Keep: retry-loops, error-hotspots, context-bloat, token-concentration, work-rhythm, tool-mix. Retune for Copilot vocabulary:

- **Cache health** → only emit if token data exists; otherwise drop the card (Copilot has no exposed cache tier).
- **Model mix** → "Everything runs on a premium model" with the premium-request angle, instead of "Everything runs on Opus."
- **Tool-mix** → key on Copilot's agent tool names (e.g. `shell` , `write` , `edit` ) rather than `Bash` .

6.6 Two-pass parse

Pass 1: VS Code chat sessions (Source A). Pass 2: Copilot CLI sessions (Source B), attributed to the same repo and de-duplicated by session id. v1 may ship Pass 1 only; the seam is already there.

## 7 · AI insights via the Copilot CLI

The "✦ Extract AI lessons" button mines real failures from the parsed history and asks the user's own Copilot CLI to write project-specific conventions. Runs on the existing Copilot subscription — no API key, no extra billing.

### 7.1 Evidence collection (port verbatim)

The original's `collect_evidence(dir, max_episodes=60)` walks the data, pairs each failed tool result with the call that produced it, and emits a compact log — one line per failure: `repo=... tool=... input=... error=...` (inputs truncated to 220 chars, errors to 300, capped at 10 per tool to avoid noise). This logic is source-agnostic once the adapter labels tool calls/errors; reuse it unchanged.

### 7.2 The CLI call — programmatic mode

GitHub Copilot CLI's programmatic mode is the direct analog of `claude -p`: pass a prompt, read clean text from stdout, process exits.

```
copilot -p "<prompt>" -s --no-ask-user --allow-tool '' --model <model>
```

| FLAG                         | WHY TOKENBOARD USES IT                                                                             |
|------------------------------|----------------------------------------------------------------------------------------------------|
| <code>-p / --prompt</code>   | Single-shot prompt, no interactive UI. (Alternatively, pipe the prompt on stdin.)                  |
| <code>-s (silent)</code>     | Suppresses session metadata → clean text on stdout, easy to parse.                                 |
| <code>--no-ask-user</code>   | Never blocks waiting for clarification — required for an unattended subprocess.                    |
| <code>--allow-tool ''</code> | This is a pure text/reasoning task. Grant <b>no</b> tools so the agent can't touch the filesystem. |
| <code>--model</code>         | Pin a cheap, fast model for a deterministic, low-cost call.                                        |

Reuse the exact prompt from the original (it already asks for a raw JSON array of `{title, convention, evidence, scope}` and forbids prose/fences), changing only "Claude Code session transcripts" → "GitHub Copilot session history" and "suitable for a CLAUDE.md file" → "suitable for a `.github/copilot-instructions.md` file."

### 7.3 Spawning on Windows (replaces the zsh path)

```
use std::process::{Command, Stdio};

// Copilot installs a `copilot.cmd` shim on PATH. Spawn via cmd so PATHEXT resolves it
// even when Tokenboard is launched from Explorer (GUI context, minimal env).
let mut child = Command::new("cmd")
    .args(["/C", "copilot -p -s --no-ask-user --allow-tool \"\""]) // "-" → read prompt from stdin
    .stdin(Stdio::piped()).stdout(Stdio::piped()).stderr(Stdio::piped())
    .spawn()?;
child.stdin.take()?.write_all(prompt.as_bytes());
let out = child.wait_with_output()?;
// strip optional ```json fences defensively, then serde_json::from_str(&cleaned)
```

**Two Windows gotchas.** (1) Resolve the binary robustly: try `copilot` via `cmd /C` first; if it fails, fall back to `where copilot` and to `gh copilot` (the older extension form). Surface the exact stderr to the UI on failure — the original learned this the hard way with "not logged in." (2) First-run auth: if the CLI returns a not-authenticated error, show a one-line fix in the UI: run `copilot` once interactively (or `gh auth login`) to sign in, then retry.

### 7.4 Failure surfacing

Mirror the original's error handling: many CLIs emit a structured error envelope on stdout even on non-zero exit. Parse stdout first for a human-readable message; fall back to stderr; never show an empty error. Map the common "not logged in" case to the actionable hint above.

## 8 · Frontend: the five-tab dashboard

The frontend is one self-contained `index.html` with an inlined Chart.js ( `chart.umd.min.js` ) and no build step. It ports unchanged except for cosmetic relabeling. It reads the contract via `window.__TAURI__.core.invoke('analyze')` and the lessons via `invoke('ai_insights')`.

### 8.1 Tab structure

| TAB              | RENDERS FROM                                                          | WIDGETS                                                                                             |
|------------------|-----------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Overview         | <code>totals</code> , <code>efficiency</code> , <code>insights</code> | 8 KPI cards · insight cards · "📌 Extract AI lessons" + "📄 Copy for copilot-instructions.md" buttons |
| Tokens / Usage   | <code>daily</code> , <code>totals</code> , <code>efficiency</code>    | daily token/premium-request line+bar chart · usage doughnut · efficiency cards                      |
| Repos & Sessions | <code>repos</code> , <code>sessions</code>                            | two sortable tables (click any header) · error pills · heaviest-sessions list                       |
| Tools & Models   | <code>tools</code> , <code>models</code>                              | horizontal stacked bar (calls vs errors) · cost-by-model doughnut + table                           |
| Rhythm           | <code>hourHist</code> , <code>dowHist</code>                          | 24-cell hour heat strip · 7-bar weekday chart                                                       |

### 8.2 Tab mechanism (already implemented)

```
// each section is <section class="tabsec" id="sec-NAME" [hidden]>
const showTab = name => {
  document.querySelectorAll('.tabsec').forEach(s => s.hidden = (s.id !== 'sec-'+name));
  document.querySelectorAll('.tab').forEach(t => t.classList.toggle('active', t.dataset.tab===name));
  localStorage.setItem('tb_tab', name);
};
showTab(location.hash.slice(1) || localStorage.getItem('tb_tab') || 'overview');
```

Hash deep-links ( `#tokens` ) double as the screenshot hook for docs/marketing — drive a headless browser to each `#tab` and capture. The original's brochure deck was built exactly this way.

### 8.3 Relabeling for Copilot

- "Notional value (\$)" → "Premium requests" (same `cost` field).
- "Cache hit rate" card → hide when token data is absent.
- "Copy for CLAUDE.md" → "Copy for copilot-instructions.md"; paste target is `.github/copilot-instructions.md` in the repo.
- Footer pricing note → describe premium-request multipliers instead of per-token rates.

Palette, Chart.js config, KPI layout, heat strip, and sortable-table code are unchanged.

## 9 · Tauri shell & Windows packaging

### 9.1 tauri.conf.json (Windows-trimmed)

```
{
  "productName": "Tokenboard",
  "version": "1.0.0",
  "identifier": "com.yourname.tokenboard",
  "build": { "frontendDist": "../src" },
  "app": {
    "withGlobalTauri": true,           // exposes window.__TAURI__ to the static page
    "windows": [{ "title": "Tokenboard", "width": 1340, "height": 920,
                  "minWidth": 800, "minHeight": 600 }],
    "security": { "csp": null }
  },
  "bundle": {
    "active": true,
    "targets": ["msi", "nsis"],       // Windows only – was "all"
    "icon": ["icons/icon.ico", "icons/icon.png"],
    "windows": {
      "webviewInstallMode": { "type": "downloadBootstrapper" } // pulls WebView2 if absent
    }
  }
}
```

Drop the macOS `signingIdentity` block and the `.icns` entry entirely. `main.rs` keeps `#![cfg_attr(not(debug_assertions), windows_subsystem = "windows")]` to suppress the console window.

### 9.2 Path resolution

```
fn appdata() -> PathBuf {
    // %APPDATA% (Roaming)
    PathBuf::from(std::env::var("APPDATA").unwrap_or_default())
}
fn code_workspace_storage() -> PathBuf {
    appdata().join("Code").join("User").join("workspaceStorage")
}
```

### 9.3 Outputs & the SmartScreen note

| ARTIFACT                       | BUILT BY               | NOTES                          |
|--------------------------------|------------------------|--------------------------------|
| Tokenboard_1.0.0_x64_en-US.msi | WiX (bundled by Tauri) | Enterprise-friendly installer  |
| Tokenboard_1.0.0_x64-setup.exe | NSIS                   | Lightweight per-user installer |

**Unsigned-app first run.** Without an EV/OV code-signing certificate, Windows SmartScreen shows "Windows protected your PC." The one-time bypass: **More info** → **Run anyway**. Document this in the release notes. Removing the prompt entirely requires a purchased Authenticode certificate added to the CI signing step — the Windows analog of the macOS notarization tax.

## 10 · CI/CD — single Windows job

---

Collapse the original 4-way matrix to one `windows-latest` runner. To avoid the duplicate-draft race the original hit, create the release once, then upload into it.

```
name: Release (Windows)
on: { push: { tags: ["v*"] }, workflow_dispatch: {} }

jobs:
  build:
    runs-on: windows-latest
    permissions: { contents: write }
    steps:
      - uses: actions/checkout@v4
      - uses: dtolnay/rust-toolchain@stable
      - uses: actions/setup-node@v4
        with: { node-version: 22 }
      - run: npm install
        working-directory: app
      - uses: tauri-apps/tauri-action@v0          # builds MSI + NSIS, creates the release
        env: { GITHUB_TOKEN: "${{ secrets.GITHUB_TOKEN }}" }
        with:
          projectPath: app
          tagName: "${{ github.ref_name }}"
          releaseName: "Tokenboard ${{ github.ref_name }}"
          releaseDraft: true
```

One runner emits both Windows installers into a single draft — no cross-platform matrix, so the split-asset race the original suffered cannot occur. Tag `v1.0.0` to trigger; review the draft; publish.

## 11 · Build & run — exact commands

### PREREQUISITES (WINDOWS DEV BOX)

```
# Rust (MSVC toolchain), Node 22, and the Tauri prerequisites
winget install Rustlang.Rustup
rustup default stable-x86_64-pc-windows-msvc
winget install OpenJS.NodeJS.LTS
winget install Microsoft.VisualStudio.2022.BuildTools # "Desktop development with C++"
# WebView2 runtime ships on Win11; the installer bootstraps it on Win10 if missing.
npm install -g @tauri-apps/cli@^2
```

### PROJECT SKELETON

```
tokenboard/
  app/
    package.json          # dev-deps: @tauri-apps/cli
    src/                  index.html  chart.umd.min.js
    src-tauri/             Cargo.toml  tauri.conf.json  build.rs
                           src/ main.rs  core/{model.rs,analyzer.rs}  ingest/{copilot.rs,probe.rs}
                           icons/ icon.ico  icon.png
```

### DEVELOP / BUILD / DEBUG

```
# live dev window
cd app && npm install && npx tauri dev

# calibrate the adapter against THIS machine's Copilot data (run first!)
cargo run --manifest-path src-tauri/Cargo.toml -- --probe # dumps distinct keys/shapes

# headless checks (no window)
cargo run ... -- --dump # prints the full analyze() JSON
cargo run ... -- --ai # tests the Copilot-CLI lesson extraction

# release installers (MSI + NSIS) → src-tauri\target\release\bundle\
cd app && npx tauri build
```

**Recommended first hour.** 1) Stand up the skeleton and confirm `npx tauri dev` opens a blank window. 2) Write `--probe` and run it — *look at the real JSON before writing the adapter*. 3) Map the 10 fields in the 4.3 table. 4) `--dump` until `totals.sessions` and `hourHist` look right. 5) Drop in the unchanged frontend. 6) Wire `copilot -p`. Ship.

## 12 · Risks, unknowns & validation

### 12.1 Known unknowns (resolve on the target machine)

| UNKNOWN                                      | IMPACT                                | HOW TO RESOLVE                                                                                           |
|----------------------------------------------|---------------------------------------|----------------------------------------------------------------------------------------------------------|
| chatSessions JSON schema & exact field names | High — the adapter depends on it      | <code>--probe</code> dumps keys; write the adapter against observed reality, not this doc                |
| Per-message timestamps present & parseable   | High — active/idle + rhythm need them | Probe a message; detect epoch-ms vs RFC3339; if absent, fall back to file mtime (coarser)                |
| Token counts present                         | Medium — Tokens tab richness          | If absent, lead with request volume + premium requests; keep keys at 0                                   |
| Premium-request multipliers                  | Medium — cost accuracy                | Read current values from GitHub billing docs at build; embed as JSON, never hardcode here                |
| Copilot CLI binary name/auth                 | Medium — AI lessons feature           | Resolve <code>copilot</code> → <code>gh copilot</code> fallback; surface "not logged in" with a fix hint |
| Agent tool-call shape in logs                | Low — Tools tab + evidence            | Probe agent sessions; if unavailable in v1, Tools tab simply shows fewer rows                            |

### 12.2 Validation checklist (definition of done for v1)

- `--probe` prints a stable key map for at least 3 real workspaces.
- `--dump : totals.sessions` equals the number of chat sessions the user recognizes; `repos[].name` are real folder names (not hashes).
- `hourHist` / `dowHist` sum to the user-turn count and peak at a plausible hour.
- `activeSeconds` > 0 and within an order of magnitude of felt usage.
- Frontend renders all five tabs with no console errors; zero-token fields degrade gracefully.
- "♦ Extract AI lessons" returns a JSON array of project-scoped conventions; "not logged in" shows the fix hint.
- `npm tauri build` produces both installers; a clean Win10 VM installs and launches (WebView2 bootstraps).
- Tag → CI emits one draft release with both artifacts.

### 12.3 Privacy & honesty guarantees to preserve

- Read-only access to Copilot data; never write to VS Code storage.
- No network calls except the user-initiated `copilot -p` subprocess (which itself rides the user's account).
- Label premium-request figures as **consumption**, not money — Copilot is subscription-based; don't imply per-use billing.
- If a metric can't be derived (e.g. no tokens), show it as unavailable rather than fabricating a number.

**Bottom line.** The frontend, analytics, insight rules, packaging, and CI are a known-good port of a shipped app. The one genuinely new, genuinely uncertain piece is reading Copilot's local format — so the whole design quarantines that uncertainty inside `ingest/copilot.rs` behind a probe-first workflow. Build the probe, look at the data, map ten fields, and the rest falls into place.

## Appendix A · File tree & dependencies

### A.1 Full tree

```
tokenboard/
├── .github/workflows/release.yml      # single windows-latest job
├── app/
│   ├── package.json                 # devDeps: @tauri-apps/cli ^2
│   └── src/
│       ├── index.html               # dashboard (5 tabs) – ported ~verbatim
│       ├── chart.umd.min.js         # inlined Chart.js, no CDN
│       └── src-tauri/
│           ├── Cargo.toml
│           ├── build.rs              # tauri-build
│           ├── tauri.conf.json       # targets: ["msi","nsis"]
│           ├── icons/ { icon.ico, icon.png }
│           └── src/
│               ├── main.rs           # commands + --dump/--ai/--probe, CLI spawn
│               ├── core/
│               │   ├── model.rs      # Stats/Session/ModelAgg/DailyAgg (verbatim)
│               │   ├── analyzer.rs   # rollups + build_insights + output (verbatim)
│               └── ingest/
│                   ├── copilot.rs    # THE adapter – Copilot → Event stream
│                   └── probe.rs      # --probe schema dumper
```

### A.2 Cargo.toml

```
[package]
name = "tokenboard"
version = "1.0.0"
edition = "2021"

[build-dependencies]
tauri-build = { version = "2", features = [] }

[dependencies]
tauri = { version = "2", features = [] }
serde = { version = "1", features = ["derive"] }
serde_json = "1"
chrono = "0.4"
# optional, only if you read the SQLite index instead of globbing JSON:
# rusqlite = { version = "0.31", features = ["bundled"] }

[profile.release]
opt-level = 2
```

### A.3 Dependency footprint

| LAYER      | DEPENDENCY          | PURPOSE                                                     |
|------------|---------------------|-------------------------------------------------------------|
| Shell      | Tauri 2 + WebView2  | Native window, ~10 MB binary, IPC to the page               |
| Backend    | serde_json          | Parse Copilot session JSON, emit the contract               |
| Backend    | chrono              | Timestamp parsing, local-time hour/weekday, active/idle     |
| Frontend   | Chart.js (vendored) | Line/bar/doughnut charts; no other JS deps, no build step   |
| AI lessons | GitHub Copilot CLI  | User-provided; spawned as a subprocess in programmatic mode |
| Packaging  | WiX + NSIS          | Bundled by tauri-action; MSI + setup.exe                    |

Sources for the Windows/Copilot specifics in this document: GitHub Docs — *About / Using / Running GitHub Copilot CLI programmatically* (docs.github.com/copilot); The GitHub Blog — *GitHub Copilot CLI 101*; VS Code Copilot Chat storage locations per microsoft/vscode and github/copilot-cli issue trackers and the github community discussions. The premium-request multiplier table must be re-confirmed against GitHub's current billing documentation at build time. All structural/analytics content is derived from the shipped Tokenboard (Claude/macOS) source.